

ARCHITECTURE OVERVIEW

1.1 Description

The system is a distributed, event-driven architecture consisting of a web frontend and multiple backend services. The backend uses a message broker for transferring data between different services. Frontend has a websocket connection to backend for transferring aggregated emote data and a REST API connection for administrative actions. [1]

The backend is split into two different planes [1]:

- Data plane, which handles emote data and aggregation
- Control plane, which handles the configurations and settings.

1.2 Diagram

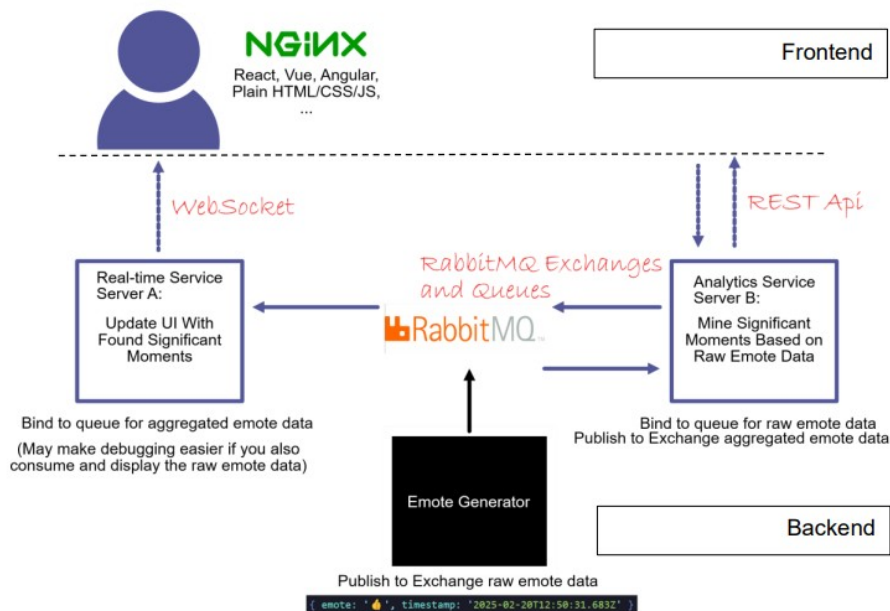


Figure 1: A high-level view of the system architecture [1][Group project assignment document. 2026]

DESIGN AND ARCHITECTURAL DECISIONS

1.3 Architectural patterns used

RabbitMQ exchanges are used to decouple emote producers from consumers which follows topics pattern.

Each backend service and frontend have their own responsibility and are set in own containers which is typical in Microservice style architecture.

1.4 Modules and technologies

1.4.1 Emote generator

Generates emote events every second. Generator has a 80% chance to produce a single emote and 20% chance to produce a burst of multiple same emotes. Object it produces contains the emote(s) and a timestamp as a ISO string.

Technology for this module's implementation will be a Node.js based application written in either Javascript or Typescript. This decision is based on developers known technologies and good libraries and support for such a backend service.

1.4.2 Rabbit MQ

Is used as a AMQP-based message broker in the application. Its main purpose is to decouple the producers and consumers of data which again enables scaling of the application. The service will be used with the topics configuration for transferring the raw emote data from emote generator and the aggregated meaningful moments from Analytics service B to Real-time service A.

Technology for this module's implementation will Be a Rabbit MQ broker service based on the project assignment.

1.5 Realtime service (Server A)

Purpose of the module is to consume the aggregated data produced by Analytics service B distributed by Rabbit MQ messaging broker. It delivers the data to the frontend via WebSockets.

Technology for this module's implementation will be a Node.js based application written in either Javascript or Typescript. This decision is based on developers known technologies and good libraries and support for such a backend service. Websockets is a new technology for the developer.

1.6 Analytics service (Server B)

Purpose of the module is to consume the raw emote events produced by the emote generator, which is then transferred to Analytics service B through Rabbit MQ messaging broker. It then parses the significant moments from the raw data and serves them to the message broker. It also has a REST API for changing settings on the service.

Technology for this module's implementation will be a Node.js based application written in either Javascript or Typescript. This decision is based on developers known technologies and good libraries and support for such a backend service. Websockets is a new technology for the developer.

1.7 Frontend

Purpose of the module is to display the significant moments in real time served by the Realtime service (Server A) using WebSocket connection. It also allows user to modify Analytics service (Server B) settings.

Technology for this module's implementation will be a Vite based React/Typescript frontend. The developer has a basic understanding of the technologies based on the Tuni's Front End course but much of it will be new. If this becomes too hard to implement the project will revert to vanilla Javascript and HTML.

1.8 Nginx

Purpose of the module is to implement a reverse-proxy for serving data from backend to frontend. As the whole application will be containerized with Docker only exposed gateway between frontend and backend will be implemented through Nginx.

Developer has small experience working with Nginx on similar projects.

1.9 Docker

Whole application will be containerized in individual service containers. A Docker compose will be implemented to orchestrate the build process. There will be two networks implemented on Docker. One for backend which will have all the backend services. Only Nginx will be exposed to host machine.

Developer has some experience with working with Docker and Docker compose from own projects.

BUILD AND RUN INSTRUCTIONS

In the project root folder run

“docker compose up --build -d” to build and start the app.

The app can be used from browser either by navigating to

“<http://localhost>” or “http://127.0.0.1”

REFERENCES

- [1] COMP.CS.510 Advanced Web Development – Back End Group project assignment document, 2026. Available:
https://moodle.tuni.fi/pluginfile.php/5681327/mod_label/intro/Group%20project%202026.pdf